DATA ENGINEERING & CLOUD · LESSON 13 OF 15

Data Architecture and System Design for Data and AI Systems

Design a data platform end to end. Describe sources, ingestion, storage, serving and governance as layers, reason about the requirements that drive each choice (freshness, scale, cost, trust, security), and apply system design basics such as bottlenecks, caching, replication and failure modes to data and AI systems.

Gap-fill lesson · ✦ 2026 · Advanced · ⏱ 150 min

By the end of this lesson you can

- Describe a data architecture with sources, ingestion, storage, serving and governance layers
- Turn business requirements into freshness, scale, cost and trust targets
- Apply system design basics to find bottlenecks and single points of failure
- Write a short architecture decision record for a key choice

Architecture starts with requirements

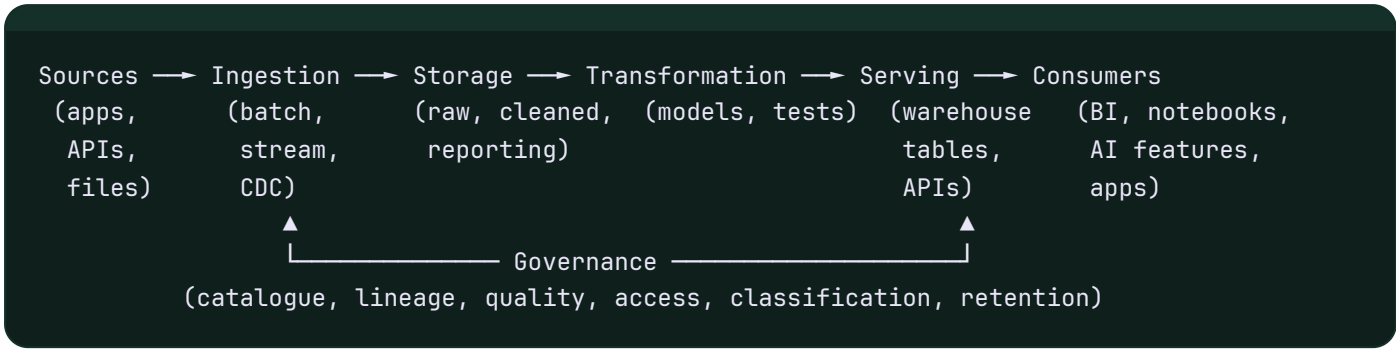
A data architecture is a set of decisions about how data moves and where it is stored. The decisions follow from requirements, and most bad architectures come from choosing tools before stating what they must do. Before drawing any diagram, write the requirements in measurable terms:

Requirement	Measurable form
Freshness	Dashboard shows data no older than 6 hours
Scale	200 million events a month, growing 30% a year
Trust	Revenue matches the ledger within 0.5% every month
Cost	Platform under £8,000 per month in year one
Security	Personal data visible only to the customer team
Availability	Reports available 99.5% of working hours

These numbers decide the architecture. A 6-hour freshness target does not need streaming; a 30-second fraud alert does.

> **CASE STUDY** **The architecture that was too fast**
>
> A team designed a streaming platform for a sales dashboard that the executives checked once a week. The platform cost four times the budget and was harder to test. A requirements review found that a daily batch pipeline met every need. The lesson is that the fastest architecture is not automatically the right one.

The layers of a data architecture



Layer	Question it answers	Typical choices
Sources	Where does the data originate?	Operational databases, SaaS APIs, event streams, files
Ingestion	How does it arrive?	Batch extracts, CDC, streaming, file drops
Storage	Where does it live, raw and modelled?	Object storage with open table formats; a warehouse
Transformation	How is it cleaned and modelled?	SQL models with tests; Spark for large jobs
Serving	How do consumers read it?	Warehouse tables, semantic models, APIs
Governance	Who may use it, and can we trust it?	Catalogue, lineage, quality checks, access control

Governance is drawn across the whole diagram because it is not a single step. Every layer needs ownership, quality rules and access rules.

System design basics for data

Data systems fail in the same ways as other distributed systems. Four ideas explain most of the design choices.

Bottlenecks. The slowest component sets the speed of the whole system. Find it by measuring each stage: the source's query capacity, the network, the warehouse's concurrency, the dashboard's query count. Improving anything other than the bottleneck does nothing.

Caching. Store the result of an expensive computation and reuse it. A reporting table that is rebuilt nightly is a cache of the detailed data. A cache is correct only when its refresh rule matches the freshness requirement, and stale caches are a classic source of wrong numbers.

Replication and backup. Copies protect against loss and let reads spread out. Backups protect against mistakes: a deleted table or a bad load. Replication alone copies the mistake; backups and time travel let you go back.

Failure modes. Ask what happens when each component fails: the source is down, the ingestion job crashes, the warehouse is slow, a schema changes upstream. A good design says, for each failure, whether data is delayed, lost, duplicated or served wrongly, and how the team will know.

Failure	Possible effect	Design response
Source unavailable	Data delayed	Retry; alert on freshness; show "last updated" on dashboards
Duplicate delivery	Totals overstated	Idempotent loads and deduplication
Schema change upstream	Pipeline breaks or silently mis-reads	Schema checks; contracts with the source team
Warehouse overloaded	Dashboards slow	Serve summaries; limit concurrency
Bad load publishes	Wrong numbers visible	Quality gate before publish; rollback

A design walk-through: a retail sales platform

Requirements. Dashboards refresh by 07:00 with yesterday's sales (batch is enough). Revenue must reconcile with the ledger monthly. Personal customer data visible only to the customer team. Budget modest.

Design. 1. Sources: the point-of-sale system (nightly file export) and the online store (API, paginated). 2. Ingestion: a nightly batch job with a watermark and a safety window (see the ingestion lesson). 3. Storage: bronze, silver and gold layers (medallion, see the storage lesson), in one lakehouse. 4. Transformation: SQL models with tests, run after ingestion, with a quality gate

before gold. 5. Serving: gold tables for dashboards; the customer team reads silver with masking applied. 6. Governance: catalogue entry per table with owner and freshness; personal fields classified and masked; monthly reconciliation to the ledger. 7. Failure responses: a missing file blocks the run and alerts the owner; a failed gate keeps yesterday's gold visible with a "data delayed" banner.

What was rejected and why. Streaming: not needed for daily reports and costly to operate. A second copy of the warehouse per team: duplicates cost and drifts; one lakehouse with access control is simpler.

Architecture decision records

An **architecture decision record (ADR)** is a short document that states one decision, the context, the options considered, the choice and its consequences. It stops the team from re-arguing a decision every year and explains the platform to new engineers.

MARKDOWN

```
# ADR 007: Use daily batch for retail sales reporting

Status: accepted (2026-01-15)
Context: dashboards need yesterday's sales by 07:00; no decision depends on intraday data.
Options: (a) daily batch; (b) hourly micro-batch; (c) streaming.
Decision: (a) daily batch, with a quality gate before publish.
Consequences: data is at most one day old; streaming can be added later if a requirement
appears;
we accept the cost of a nightly run and a 06:00 deadline.
```

INTERVIEW CORNER

How would you design a data platform for a company that is just starting?

Start with three requirements: what decisions the data supports, how fresh it must be, and who may see what. Choose the simplest architecture that meets them: batch ingestion into a lakehouse or warehouse, medallion layers, tested SQL models, one catalogue and access control. Write an ADR for each major choice and add streaming or extra platforms only when a measured requirement needs them.

::::key What to remember - Write measurable requirements before choosing tools; they decide the architecture. - Layers: sources, ingestion, storage, transformation, serving and consumers, with governance across all of them. - System design basics: find the bottleneck, cache with the right refresh rule, replicate and back up, and plan for each failure. - Keep the simplest design that meets the requirements, and record decisions in ADRs. ::::

Quick quiz

5 questions · tap an answer to check it

1. What should drive the choice between batch and streaming?

- **A** The newest technology
- **B** The measured freshness requirement and the cost of meeting it
- **C** The number of engineers
- **D** The colour of the dashboard

Answer: B. Freshness requirements decide whether streaming is worth its cost.

2. Why are backups needed even when data is replicated?

- **A** Replicas cannot be read
- **B** Replication copies mistakes too; backups and time travel allow recovery to an earlier state
- **C** Backups are cheaper than storage
- **D** Replicas are always out of date

Answer: B. Replication does not protect against bad data.

3. What should a design do when a quality gate fails before publishing?

- **A** Publish anyway to be safe
- **B** Keep the last good data visible and alert the owner with a delay notice
- **C** Delete the source
- **D** Stop all reporting permanently

Answer: B. Failing safe keeps trustworthy numbers visible.

4. What is the main purpose of an architecture decision record?

A To replace code tests

B To record a decision, its context and its consequences so it is not re-argued unnecessarily

C To store passwords

D To describe colours

Answer: B. ADRs preserve the reasoning behind decisions.

5. Improving a component that is not the bottleneck will:

A Speed up the whole system

B Change little, because the bottleneck still limits overall speed

C Always cause failures

D Remove the need to measure

Answer: B. The slowest component sets the overall pace.

EXERCISE **Design and record one decision**

For a company that needs a customer churn dashboard refreshed weekly and personal data protected, draw the six-layer architecture, state three measurable requirements, and write one ADR for the storage choice (warehouse, lakehouse or both), with the context, options, decision and consequences.

Show solution (worked answer)

Expected: a weekly batch pipeline, medallion layers, masking before the consumer layer, and an ADR that chooses the simplest store that meets the reporting and governance requirements, naming the trade-off accepted.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.